



Scaling Your Applications

AutoScalers & Metrics

Intermediate Kubernetes — Module 2 of 13



Module Objectives

- Understand why resource requests and limits are the foundation of scaling
- Explain how the Metrics Server collects and serves resource data
- Configure Horizontal Pod Autoscaler (HPA) with the `autoscaling/v2` API
- Calculate expected scaling using the HPA algorithm
- Tune scaling behavior with stabilization windows and policies
- Understand VPA and Cluster Autoscaler, and when KEDA is appropriate
- Monitor scaling with `kubectl top` and Prometheus

Why Scaling Matters

The cost of getting it wrong

The Scaling Challenge

Over-provisioned

- Wasted compute and higher cloud costs
- Resources unavailable to other teams
- Quota exhaustion on the cluster

Under-provisioned

- Degraded response times and timeouts
- OOMKilled pods
- Customer-facing impact

The goal: Right-size automatically based on real demand, not guesswork.



Resource Requests & Limits

The foundation everything else depends on



Requests vs. Limits

Requests (Guaranteed)

- What the scheduler uses for placement
- Guaranteed minimum resources
- Used by HPA for percentage calculations
- **Must** be set for HPA to work

Limits (Maximum)

- Enforced ceiling the container cannot exceed
- CPU: throttled when exceeded
- Memory: OOMKilled when exceeded
- Protects neighbors on the same node

```
resources:
  requests:
    cpu: "250m"      # 0.25 CPU cores guaranteed
    memory: "256Mi"  # 256 MiB guaranteed
  limits:
    cpu: "500m"      # Can burst to 0.5 cores, then throttled
    memory: "512Mi"  # Killed if exceeds 512 MiB
```

12
34

Understanding Resource Units

CPU Units

Value	Meaning
1	1 vCPU / 1 core
500m	Half a core (500 millicores)
250m	Quarter core
100m	1/10th of a core

1000m = 1 CPU. Always compressible.

Memory Units

Value	Meaning
128Mi	128 mebibytes (2^{20})
256Mi	256 MiB
1Gi	1 gibibyte (2^{30})
512M	512 megabytes (10^6)

Use Mi/Gi (binary). Not compressible.

Quality of Service (QoS) Classes

Kubernetes assigns a **QoS class** based on resource configuration. Under memory pressure, the kubelet evicts lower-priority QoS classes first.

QoS Class	Criteria	Eviction Priority
Guaranteed	requests == limits for all containers	Last to be evicted
Burstable	requests < limits (or only requests set)	Evicted after BestEffort
BestEffort	No requests or limits set	First to be evicted

QoS Classes — Examples

```
# Guaranteed — requests == limits
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

```
# Burstable — requests < limits
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1000m"
    memory: "512Mi"
```

```
# BestEffort — no resources specified (not recommended)
containers:
- name: worker
  image: myapp:latest
# No resources block → QoS = BestEffort
```



Metrics Server

The data pipeline for autoscaling



Metrics Server Architecture

How metrics flow through the system

1. **kubelet** collects resource usage from the container runtime (containerd)
2. **Metrics Server** scrapes kubelet's `/metrics/resource` endpoint on a short, configurable interval (15s by default)
3. **Metrics Server** aggregates and serves data via the `metrics.k8s.io` API
4. **HPA controller** queries `metrics.k8s.io` every 15 seconds (default)
5. **HPA controller** calculates desired replicas and patches the Deployment

```
# Verify Metrics Server is running
kubectl get deployment metrics-server -n kube-system

# Check metrics API availability
kubectl get apiservice v1beta1.metrics.k8s.io

# View resource usage
kubectl top nodes
kubectl top pods -n my-namespace
```



Monitoring with kubectl top

```
# Node resource usage
$ kubectl top nodes
NAME                CPU(cores)   CPU%   MEMORY   MEMORY%
ip-10-0-1-100.ec2   450m         22%    2847Mi   36%
ip-10-0-2-200.ec2   1200m        60%    5120Mi   65%

# Pod resource usage (current namespace)
$ kubectl top pods
NAME                CPU(cores)   MEMORY(bytes)
myapp-7d4b8c6f5-abc12  45m          128Mi
myapp-7d4b8c6f5-def34  52m          135Mi
myapp-7d4b8c6f5-ghi56  38m          122Mi
```

Horizontal Pod Autoscaler

The core scaling mechanism



The HPA Algorithm

Core Formula

$$\text{desiredReplicas} = \lceil \text{currentReplicas} \times \left(\frac{\text{currentMetric}}{\text{desiredMetric}} \right) \rceil$$

Example Calculation

Parameter	Value
Current replicas	3
CPU request per pod	250m
Current average CPU usage	200m per pod (80%)
Target CPU utilization	50%
Calculation	$\lceil 3 \times (80 / 50) \rceil = \lceil 4.8 \rceil = 5 \text{ replicas}$

HPA Algorithm — Edge Cases

- **Tolerance zone ($\pm 10\%$):** HPA won't scale if the ratio is within 0.9–1.1 of the target. This prevents flapping.
- **Missing metrics:** Pods with missing metrics are treated conservatively — scale-down assumes 100% utilization, scale-up assumes 0%.
- **Unready pods:** Excluded from the metric average during scale-up (included for scale-down).
- **Multiple metrics:** HPA evaluates each metric independently and takes the *maximum* desired replica count.

The 10% tolerance band is critical for stability. If your target is 50% CPU and current usage is between 45%–55%, the HPA takes no action. This prevents constant scaling oscillation.



HPA with autoscaling/v2 API

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 60 }
  - type: Resource
    resource:
      name: memory
      target: { type: Utilization, averageUtilization: 75 }
```


HPA: Custom Metrics & Behavior Policies

Custom Metrics

```
# Key section: metrics (inside HPA spec)
metrics:
- type: Resource           # Standard CPU metric
  resource:
    name: cpu
    target: { type: Utilization, averageUtilization: 60 }
- type: Pods               # Custom: requests/sec per pod
  pods:
    metric: { name: http_requests_per_second }
    target: { type: AverageValue, averageValue: "1000" }
- type: External           # SQS queue depth
  external:
    metric: { name: sqs_queue_length }
    target: { type: Value, value: "100" }
```

Behavior Policies

```
# Key section: behavior (inside HPA spec)
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
    - { type: Percent, value: 100, periodSeconds: 60 }
    - { type: Pods, value: 10, periodSeconds: 60 }
    selectPolicy: Max
  # scaleDown: ...
```



Stabilization Windows Explained

How Stabilization Works

The HPA examines all computed replica counts within the window and selects the highest.

Scale-Up Window

- Default: **0 seconds** (immediate)
- Selects *highest* recommendation in window
- Set higher to smooth transient spikes

Scale-Down Window

- Default: **300 seconds** (5 min)
- Selects *highest* recommendation in window
- Increase for slow-ramp-down workloads

Scaling Timeline — What Actually Happens

From spike to scaled (typical timeline)

Time	Event	Detail
T+0s	Traffic spike begins	CPU usage increases on existing pods
T+15s	Metrics Server scrapes	New metrics collected from kubelets
T+30s	HPA evaluates	Reads new metrics, calculates desired replicas
T+31s	HPA patches Deployment	Sets new replica count
T+32s	Scheduler assigns node	Pod scheduled to a node with capacity
T+35s	Image pull (if cached)	Container runtime pulls/uses cached image
T+40s	Container starts	Application begins initialization
T+55s	Readiness probe passes	Pod added to Service endpoints
T+55s	Traffic served	New pod handling requests



Monitoring HPA Status

```
# View HPA status – TARGETS shows current/target for each metric
```

```
$ kubectl get hpa myapp-hpa
```

NAME	REFERENCE	TARGETS	REPLICAS
myapp-hpa	Deployment/myapp	45%/60%,62%/75%	5

```
# Detailed conditions – is scaling working?
```

```
$ kubectl describe hpa myapp-hpa
```

```
Conditions:
```

AbleToScale	True	ReadyForNewScale
ScalingActive	True	ValidMetricFound
ScalingLimited	False	DesiredInRange

```
# View scaling events history
```

```
$ kubectl get events --field-selector involvedObject.name=myapp-hpa
```



Vertical Pod Autoscaler

Right-sizing pod resources



VPA — Vertical Pod Autoscaler

What VPA does

VPA automatically adjusts CPU and memory **requests and limits** for pods based on observed usage patterns.

Recommender

Watches resource usage and computes optimal requests/limits

Updater

Evicts pods that need resource updates (triggers restart with new values)

Admission Controller

Mutates new pods to use recommended resources on creation

VPA Operating Modes

Mode	Recommends	Applies on Create	Evicts to Update	Use Case
Off	✓	✗	✗	Dry run — observe recommendations
Initial	✓	✓	✗	Apply on new pods only
Auto	✓	✓	✓	Fully automatic (restarts pods)

Example Configuration

```
# Key section: VPA spec
spec:
  targetRef: { apiVersion: apps/v1, kind: Deployment, name: myapp }
  updatePolicy:
    updateMode: "Off" # Start with Off to observe
  resourcePolicy:
    containerPolicies:
    - containerName: myapp
      minAllowed: { cpu: "100m", memory: "128Mi" }
      maxAllowed: { cpu: "2", memory: "4Gi" }
```

⚠️ VPA + HPA — Compatibility

Do NOT use VPA (Auto mode) and HPA on the same metrics (CPU/memory) simultaneously. They will fight each other.

❌ Conflicting Setup

- HPA scales on CPU utilization
- VPA adjusts CPU requests
- VPA changes requests → HPA sees different utilization → scales differently → VPA readjusts...

✅ Compatible Setup

- HPA scales on custom metrics (RPS, queue depth)
- VPA manages CPU/memory requests
- Each autoscaler owns different dimensions
- Or: VPA in **Off** mode for recommendations only

Cluster Autoscaler

Scaling the nodes, not just the pods

Cluster Autoscaler — Node-Level Scaling

HPA and VPA scale pods — but pods need nodes to run on

When pods stay **Pending** because no node has enough CPU/memory, the Cluster Autoscaler adds nodes. When nodes sit underutilized, it removes them.

+ Scale Up

- Watches for Pending pods that fail scheduling on a resource shortfall
- Adds nodes to an EC2 Auto Scaling Group / managed node group

Respects node group min/max

— Scale Down

- Removes nodes left underutilized for ~10 minutes
- Reschedules their pods onto the remaining nodes
- Skips nodes whose pods cannot be safely evicted (respects PDBs)



Kubernetes Event-Driven Autoscaling

KEDA — Event-Driven Autoscaling

What KEDA adds beyond HPA

KEDA is a lightweight component that can scale Kubernetes workloads based on **external event sources** — and can scale to **zero**.

KEDA Capabilities

- Scale to/from zero replicas
- 60+ built-in scalers (SQS, Kafka, Prometheus, Cron)
- Works alongside native HPA via ScaledObject/ScaledJob

When to use KEDA

- Queue-based workers (SQS, Kafka)
- Schedule-based or scale-to-zero scenarios
- Complex multi-metric scaling logic



KEDA ScaledObject Example

```
# Key section: ScaledObject spec
spec:
  scaleTargetRef:
    name: sqs-consumer      # Deployment to scale
  minReplicaCount: 0        # Can scale to zero!
  maxReplicaCount: 50
  triggers:
  - type: aws-sqs-queue
    metadata:
      queueLength: "5"      # Target 5 messages per pod
  # ...
```

Scale-to-zero: When the queue is empty, KEDA scales to 0 replicas. When messages arrive, it scales back up -- reducing costs for bursty workloads.



Monitoring Scaling with Prometheus

Observability for scaling decisions

Key Prometheus Metrics for Scaling

Metric	Source	Use
<code>container_cpu_usage_seconds_total</code>	cAdvisor	Actual CPU consumption per container
<code>container_memory_working_set_bytes</code>	cAdvisor	Memory usage (what K8s uses for OOM decisions)
<code>kube_pod_container_resource_requests</code>	kube-state-metrics	Configured resource requests
<code>kube_horizontalpodautoscaler_status_current_replicas</code>	kube-state-metrics	Current HPA replica count
<code>kube_horizontalpodautoscaler_status_desired_replicas</code>	kube-state-metrics	HPA desired replica count

Scaling Best Practices

Do

- Always set resource requests
- Start with CPU-based HPA at 60-70% target
- Use conservative scale-down (300s+ window)
- Monitor HPA events and load test to validate
- Use VPA in Off mode for right-sizing data

Don't

- Skip resource requests (HPA won't work)
- Set target too high (90%+) or too low (20%)
- Use VPA Auto + HPA on same metric
- Set maxReplicas too low for actual demand
- Ignore readiness probes or cluster capacity



Recommended HPA Pattern

```
# Complete HPA with tuned behavior
spec:
  # ...scaleTargetRef, minReplicas: 3, maxReplicas: 30
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
  behavior:
    scaleUp:
      policies:
        - { type: Percent, value: 50, periodSeconds: 60 }
      selectPolicy: Max
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - { type: Percent, value: 10, periodSeconds: 120 }
      selectPolicy: Min
```

Lab 2: Configuring Autoscaling

Hands-on: HPA, VPA, and Cluster Autoscaler

- Deploy an application with resource requests and limits
- Create an HPA on CPU, generate load, and observe scaling
- Apply a v2 HPA and tune behavior policies (stabilization windows)
- Configure a VPA in Off mode and review its recommendations
- Trigger unschedulable pods and watch the Cluster Autoscaler add nodes
- Monitor with `kubectl top` and `kubectl get hpa -w`

Duration: ~45 minutes



Module 2 — Key Takeaways

- **Resource requests are the foundation** — HPA can't work without them
- **Metrics Server** provides real-time CPU/memory data for the HPA algorithm
- **HPA algorithm:** $\text{desiredReplicas} = \lceil \text{currentReplicas} \times (\text{current}/\text{desired}) \rceil$ with $\pm 10\%$ tolerance
- **autoscaling/v2** supports multiple metrics, custom metrics, and behavior policies
- **Scale up fast, scale down slow** — the golden rule of behavior tuning
- **VPA** right-sizes pods (Off mode with HPA); **KEDA** enables scale-to-zero
- **~55+ seconds** from spike to new pod serving traffic — plan headroom accordingly

? Questions & Discussion

What scaling challenges have you encountered with your current workloads?

Up Next: Module 3 — Managing Data — Storage, PVCs & StatefulSets